ENTERPRISE SYSTEM SPECIFICATION & WORKFLOW BLUEPRINT

MONEY CARD SYSTEM

COMPLETE SYSTEM WORKFLOW & TECHNICAL ARCHITECTURE

End-to-End Workflow Reference: Super Admin • Organization Admin • Staff Flutter App • Backend • PostgreSQL

Document Version: 1.0.0 (Release Candidate)	**Specification:** Money Card Core M0 V10 Multi-Tenant Audited
Release Date: August 19, 2026	**Target Audience:** Developers, Testers, Management, DevOps
Client Applications: React Web Portal + Flutter Android Staff POS	**Core Services:** Node.js/TypeScript REST API + PostgreSQL 5432 (Prisma)
Security Standard: JWT (15m Access / 7d Refresh) + Cryptographic QR	**Isolation Model:** Strict Multi-Tenant Organization & Branch Partitioning

EXECUTIVE SUMMARY & OPERATIONAL MANDATE

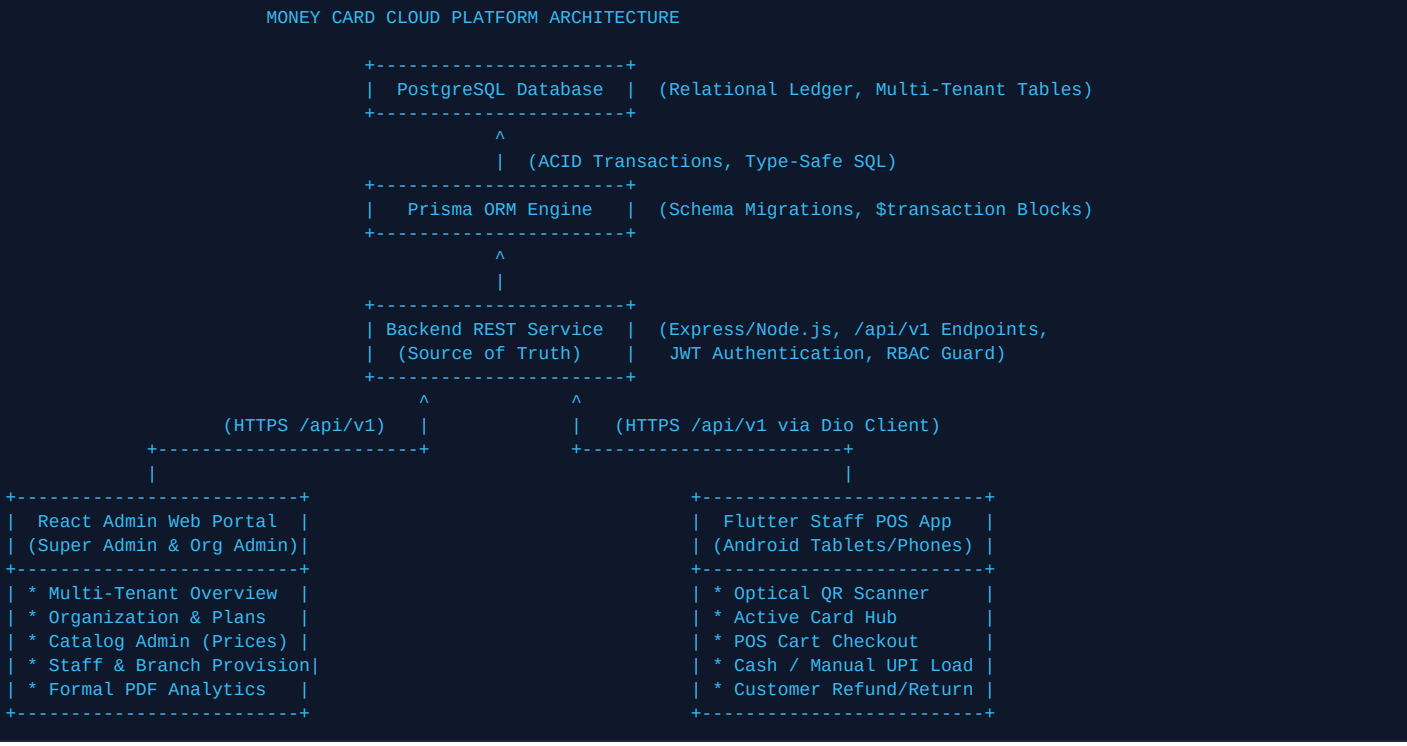
This technical document provides the authoritative, exhaustive end-to-end architectural and operational specification for the Money Card platform. It details all user roles, state lifecycles, cryptographic QR token resolution, dynamic card session accounting, double-entry financial streams (CASH and manual UPI), multi-tenant isolation barriers, automated API contracts (58 endpoints), and hybrid Mock/Real API operational modes. No functionality described herein is speculative; all workflows correspond strictly to the production codebase and M0 V10 shared contract.

DOCUMENT TABLE OF CONTENTS & SECTION INDEX

SECTION 1 — SYSTEM OVERVIEW & ARCHITECTURE

The Money Card platform is a multi-tenant, cloud-enabled closed-loop smart card and Point of Sale (POS) ecosystem designed for commercial food courts, corporate cafeterias, university dining halls, and multi-branch retail environments. The system replaces cash transactions at merchant counters with temporary, prepaid physical QR cards, providing microsecond checkout speeds, zero internet dependency at customer checkout points, and mathematical reconciliation across branches.

FLOW / ARCHITECTURE: MULTI-TIER ENTERPRISE SYSTEM ARCHITECTURE



Core Tier Breakdown & Operational Roles:

- **PostgreSQL (Port 5432):** The authoritative ACID relational data store. Stores immutable financial ledgers, tenant organizations, branch hierarchies, catalog items, physical card mappings, active card sessions, inventory balances, and cryptographically signed audit logs.
- **Prisma ORM Layer:** Provides schema definition, migration management, and type-safe query execution. Critical financial operations (such as POS purchases and card settlement) utilize Prisma atomic $transaction blocks to guarantee zero partial-state writes.
- **Backend RESTful Service (/api/v1):** The single source of truth for the entire platform. Handles JWT authentication, RBAC authorization, organization/branch isolation enforcement, business logic execution, and financial calculation validation.
- **React Admin Web Portal:** Browser-based management interface built with Vite, TypeScript, and Tailwind CSS. Serves Super Admin (platform-wide oversight) and Organization Admin (tenant management, branch network, staff provisioning, catalog creation, PDF analytics).
- **Flutter Staff Mobile Application:** High-performance Android POS client built with Flutter, Riverpod, and Dio. Designed for fast counter operations: optical QR card scanning, balance inquiries, product cart selection, cash/UPI recharges, and card settlement.

CORE ARCHITECTURAL INVARIANT

Architectural Rule: Single Source of Truth & Zero Peer-to-Peer Communication. The React Web Portal and Flutter Staff POS Application never communicate directly with each other. Both clients interact strictly as stateless HTTP consumers with the Node.js/Prisma backend over the unified /api/v1 contract. The backend database is the sole authoritative state machine for card balances, inventory stock, and session validity.

SECTION 2 — USER ROLES & ACTOR PROFILES

The Money Card system implements a Role-Based Access Control (RBAC) model defining four distinct actors. Each role operates within strict cryptographic boundaries and functional scopes.

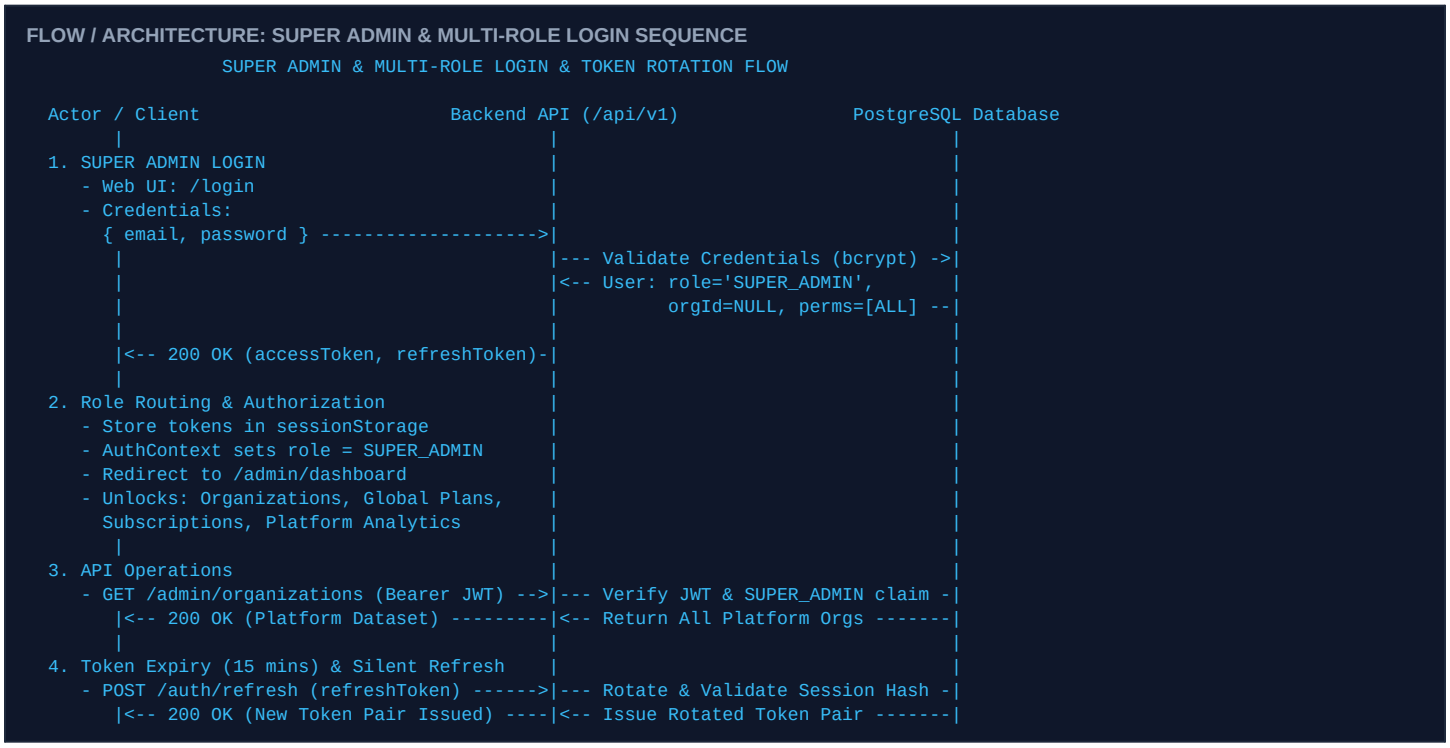
ROLE	ACTOR SCOPE	PRIMARY APPLICATION	CORE RESPONSIBILITIES & PERMITTED OPERATIONS

SUPER_ADMIN	Platform-Wide (Global Multi-Tenant)	React Web Admin /admin/*	Creates tenant organizations, provisions initial Org Admin credentials, manages global subscription plan tiers, monitors gateway subscription payments, reviews plan change requests, and exports platform-wide 3-page PDF analytics.
ORG_ADMIN	Tenant Scope (Single Organization)	React Web Admin /portal/*	Manages tenant branches, creates staff accounts, assigns granular permissions, creates product catalog items and prices, oversees branch inventory stock levels, tracks multi-branch performance, and exports organization PDF reports.
STAFF	Branch Scope (Assigned Branches)	Flutter Android App Mobile POS	Performs counter operations: scans customer QR cards, activates card sessions, builds POS shopping carts from existing branch products, executes recharges (Cash / manual UPI), processes refunds and card returns, and adjusts inventory (if permitted).
USER (Public)	Cardholder Scope (Session Token)	Public Web Portal /c/:token	Unauthenticated customer view: resolves physical card QR token to display live remaining card balance, active session duration, detailed itemized transaction history, and digital sales receipts.

SECTION 3 — AUTHENTICATION WORKFLOW & TOKEN LIFECYCLE

Authentication across all client applications utilizes a dual-token JSON Web Token (JWT) architecture. Credentials are validated against bcrypt-hashed passwords stored in PostgreSQL. The platform supports dedicated login workflows for Super Admin, Organization Admin, and Counter Staff, each routing to their respective authorization scopes.

FLOW / ARCHITECTURE: SUPER ADMIN & MULTI-ROLE LOGIN SEQUENCE



Dedicated Login Workflows by Actor Role:

USER ROLE	ACCESS CLIENT	CREDENTIALS & TOKEN PAYLOAD	POST-LOGIN ROUTING & SCOPE UNLOCKED
SUPER_ADMIN	React Web Admin /login	Platform email + password. JWT Payload: role: 'SUPER_ADMIN', organizationId: null (Global Scope).	Redirects immediately to /admin/dashboard. Unlocks platform-wide management: Create Organizations, Global Plans, Subscriptions, Platform Analytics, and Settings.
ORG_ADMIN	React Web Admin /login	Tenant admin email + password. JWT Payload: role: 'ORG_ADMIN', organizationId: '<org-id>'.	Redirects to /portal/dashboard. Scoped strictly to their single organization: Branches, Staff, Products, Cards, Inventory, and Tenant Analytics.
STAFF	Flutter Mobile App Login Screen	Staff email + password. JWT Payload: role: 'STAFF', assignedBranchIds: [...], permissions: [...].	Redirects to Mobile POS Counter Hub. Unlocks active branch selector, QR card scanner, POS cart checkout, Cash/UPI recharge, and card settlements.

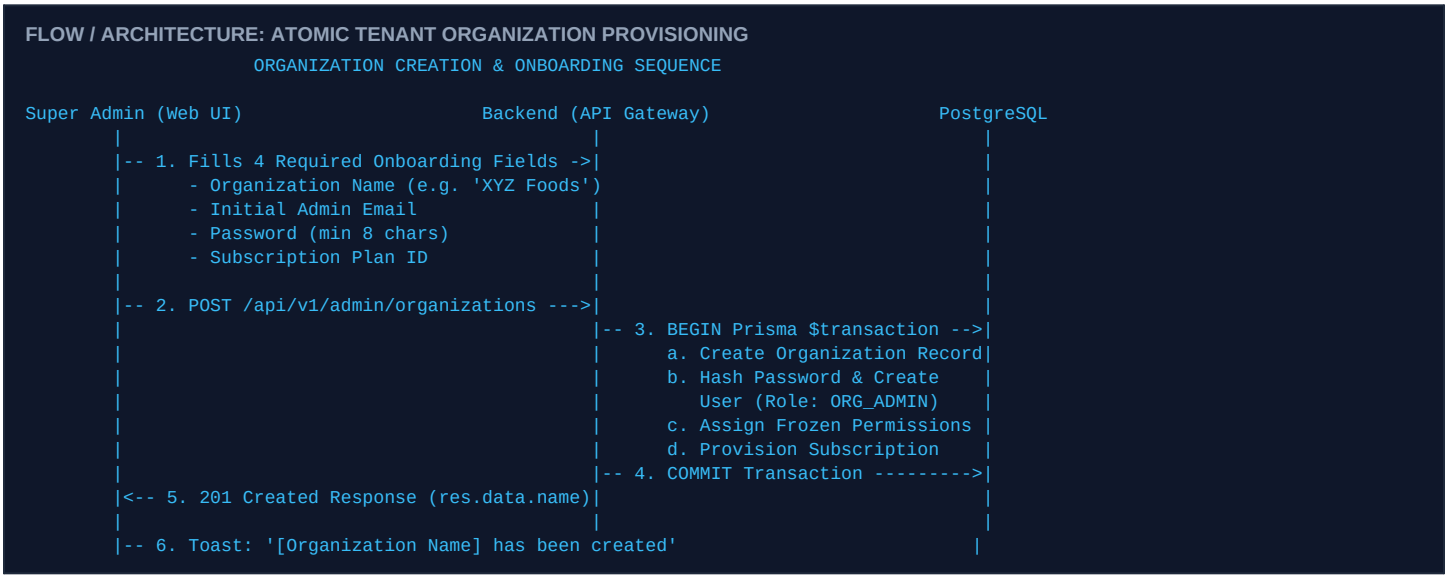
Token Storage, Expiry & Route Protection:

- **Super Admin Route Protection:** React Router enforces RoleGuard(['SUPER_ADMIN']) on all /admin/* routes. Unauthorized attempts by unauthenticated users redirect to /login; attempts by Org Admins or Staff yield HTTP 403 Forbidden.
- **Access Token Lifetime:** 15 Minutes. Encodes user UUID, role, organizationId, assignedBranchIds, and permission string array. Cryptographically signed using HMAC-SHA256 (HS256).
- **Refresh Token Lifetime:** 7 Days. Opaque cryptographically random string stored as a SHA-256 hash in the database. Rotated on every single refresh invocation to prevent replay attacks.
- **Flutter Mobile Storage:** Stored securely via flutter_secure_storage utilizing Android EncryptedSharedPreferences (backed by Hardware Keystore) and iOS Keychain Services.
- **Web Browser Storage:** Stored in memory / sessionStorage with automatic silent refresh interceptors managed via Axios request/response middleware.
- **Automatic Session Invalidation:** Calling POST /api/v1/auth/logout immediately blacklists and deletes the refresh token session from PostgreSQL, rendering all active client sessions invalid.

SECTION 4 — ORGANIZATION CREATION & ONBOARDING WORKFLOW

Tenant onboarding is strictly governed by Super Admin. Organizations are provisioned alongside an initial Organization Admin account and subscription tier assignment within an atomic transaction.

FLOW / ARCHITECTURE: ATOMIC TENANT ORGANIZATION PROVISIONING



SPECIFICATION RULE: DYNAMIC ONBOARDING NOTIFICATION

Mandatory UI Success Toast Format: Upon successful creation, the frontend MUST display the dynamic confirmation: '[Organization Name] has been created' (e.g. *'XYZ Foods has been created'*). Generic messages such as *'Organization created successfully'* or hardcoded names are strictly prohibited by the M0 specification.

SECTION 5 — ORG ADMIN WORKFLOW & TENANT BOUNDARY

Once onboarded, the Organization Admin manages their enterprise across seven operational modules: Dashboard Overview, Branch Management, Staff Administration, Product Catalog, Inventory Control, Physical Cards, and Tenant Analytics. Every database query executed by an Org Admin is scoped by the authenticated JWT claim WHERE organizationId = :tenantId.

- **Dashboard Overview:** Real-time KPI cards displaying Total POS Revenue, Wallet Recharge Volume, Net Transactions, and Active Card Sessions across all permitted branches.
- **Branch Management:** Creates operational branches (e.g., 'Downtown Food Court', 'Campus Kiosk'), updates operational status (ACTIVE / INACTIVE), and configures location metadata.
- **Staff Administration:** Provisions staff accounts, sets login credentials, maps staff members to one or more physical branches, and toggles granular operational permissions.
- **Product Catalog Administration:** Defines product catalog items, multi-select categories (e.g., ['Fast Food', 'Beverages']), and retail prices. Prices defined here are strictly authoritative.

- **Branch Inventory Oversight:** Monitors physical stock counts per product per branch. Identifies low-stock items and performs manual quantity adjustments with audited reason codes.
- **Card & Session Oversight:** Views pre-printed physical card inventory, tracks active customer sessions, and audits transaction ledgers.
- **Organization PDF Analytics:** Generates and downloads formal, multi-page PDF analytics reports filtered by branch and custom date ranges.

SECTION 6 — STAFF MANAGEMENT & FROZEN 20 PERMISSIONS MODEL

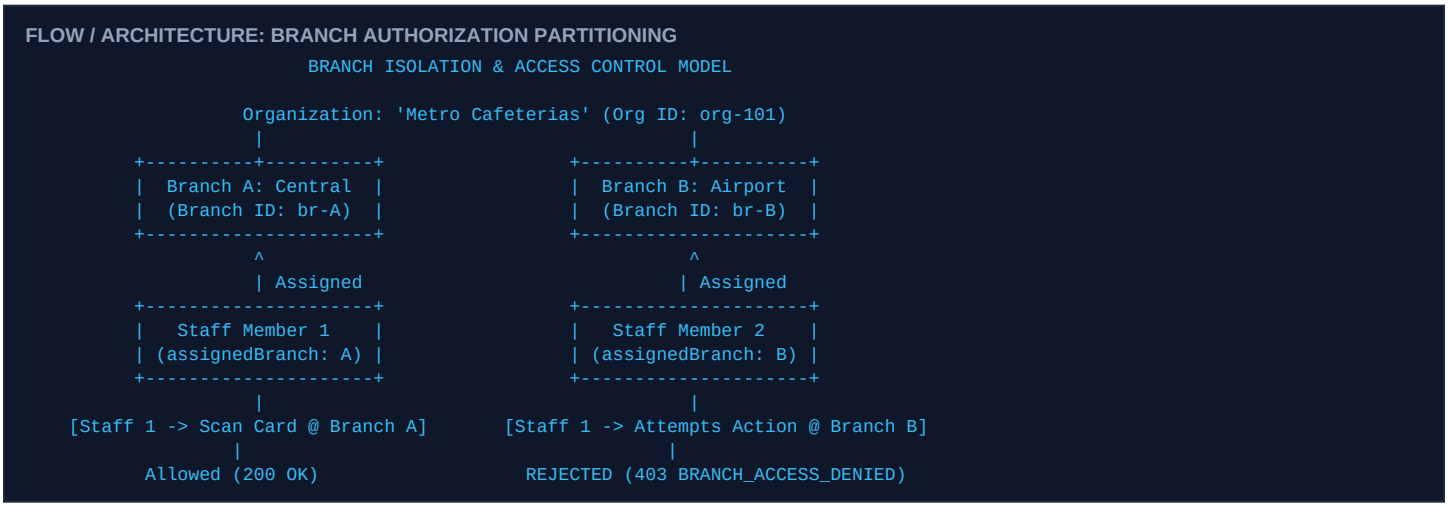
Staff members are provisioned by the Organization Admin. Access control is strictly decoupled from job titles and enforced through a frozen list of exactly 20 granular permissions defined in M0 Section 3.

DOMAIN	PERMISSION CODE	FUNCTIONAL DESCRIPTION & PERMITTED ACTION
Card Operations	CARD_VIEW	View card details, physical numbers, and QR registration status.
	CARD_ISSUE	Issue new physical cards and register unmapped QR codes.
	CARD_RETURN	Process customer card returns, calculate refund balances, and reset cards.
	CARD_BLOCK	Mark compromised or lost cards as BLOCKED.
	CARD_UNBLOCK	Restore BLOCKED cards back to AVAILABLE status.
Sessions & Payments	RECHARGE	Credit active card balances via Cash or manual UPI deposit.
	PURCHASE	Process POS checkout and deduct product costs from active sessions.
	REFUND	Execute transaction reversals and remaining balance cash refunds.
	SESSION_VIEW	Inspect active session balances, durations, and ledger records.
Products & Inventory	PRODUCT_VIEW	Browse and search catalog products within assigned branches.
	PRODUCT_MANAGE	Create and edit catalog products and prices (Admin portal only).
	INVENTORY_VIEW	View branch stock levels, quantities, and low stock alert states.
	INVENTORY_MANAGE	Adjust inventory quantities and log stock movements.
	INVENTORY_IMPORT	Bulk import inventory stock via 3-column CSV spreadsheet.
Analytics & Reports	VIEW_ANALYTICS	Access operational transaction metrics and branch comparison charts.
	VIEW_REPORTS	Generate and download formal PDF business reports.
Staff & Organization	STAFF_VIEW	List organization staff members and branch assignments.
	STAFF_MANAGE	Create staff, update assignments, and modify permission sets.
Branch Network	BRANCH_VIEW	List branch locations and inspect operational status.
	BRANCH_MANAGE	Create branches and update branch operational parameters.

SECTION 7 — BRANCH HIERARCHY & BRANCH ISOLATION WORKFLOW

Organizations operate as multi-branch networks. Staff members are explicitly mapped to one or more permitted branches. The system enforces branch isolation at both the frontend routing level and the authoritative backend middleware level.

FLOW / ARCHITECTURE: BRANCH AUTHORIZATION PARTITIONING



SECURITY PRINCIPLE: AUTHORITATIVE BRANCH ISOLATION

Dual-Layer Security Guard: Hiding a button in the Flutter UI or Web interface is never considered sufficient security. Every API request that touches branch inventory, transactions, or card sessions validates that the authenticated staff user's assignedBranchIds array contains the target branchId. Unauthorized attempts fail immediately with HTTP 403 Forbidden.

SECTION 8 — PRODUCT / CATALOG ADMINISTRATION VS POS CART

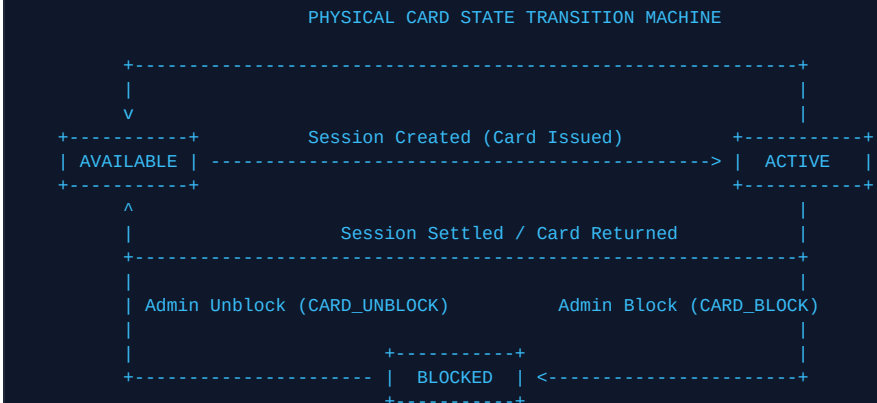
A critical architectural distinction exists between catalog administration (creating products and establishing retail prices) and POS counter operations (adding existing catalog items to a customer's active shopping cart).

ATTRIBUTE / SCOPE	'ADD PRODUCT' — CATALOG ADMINISTRATION	'ADD PRODUCTS' — POS COUNTER CART
Target Application	React Web Admin Portal (/products)	Flutter Mobile Staff App (Active Card Hub)
Authorized Actor	Organization Admin (PRODUCT_MANAGE)	Counter Staff (PURCHASE / PRODUCT_VIEW)
Core Action	Creates master product record: Name, Category Array (e.g. ['Meals', 'Combos']), and Authoritative Retail Unit Price.	Queries active branch catalog, filters by category, increments item quantities, and builds a temporary POS cart.
Price Modification	Full authority to set, discount, or update product unit prices in PostgreSQL.	**STRICTLY PROHIBITED.** Staff cannot alter, override, or discount product unit prices during checkout.
Database Mutation	Inserts into products and initializes inventory_items.	Does not modify catalog; executes atomic purchase transaction against card session.

SECTION 9 — PHYSICAL CARD LIFECYCLE & STATE MACHINE

Physical cards are reusable plastic or paper cards printed with a human-readable identifier (e.g. MC-001) and an opaque QR code URL (e.g. https://moneycard.app/c/token-xyz). Cards represent temporary account tokens that cycle through three discrete states.

FLOW / ARCHITECTURE: CARD STATE TRANSITION GRAPH

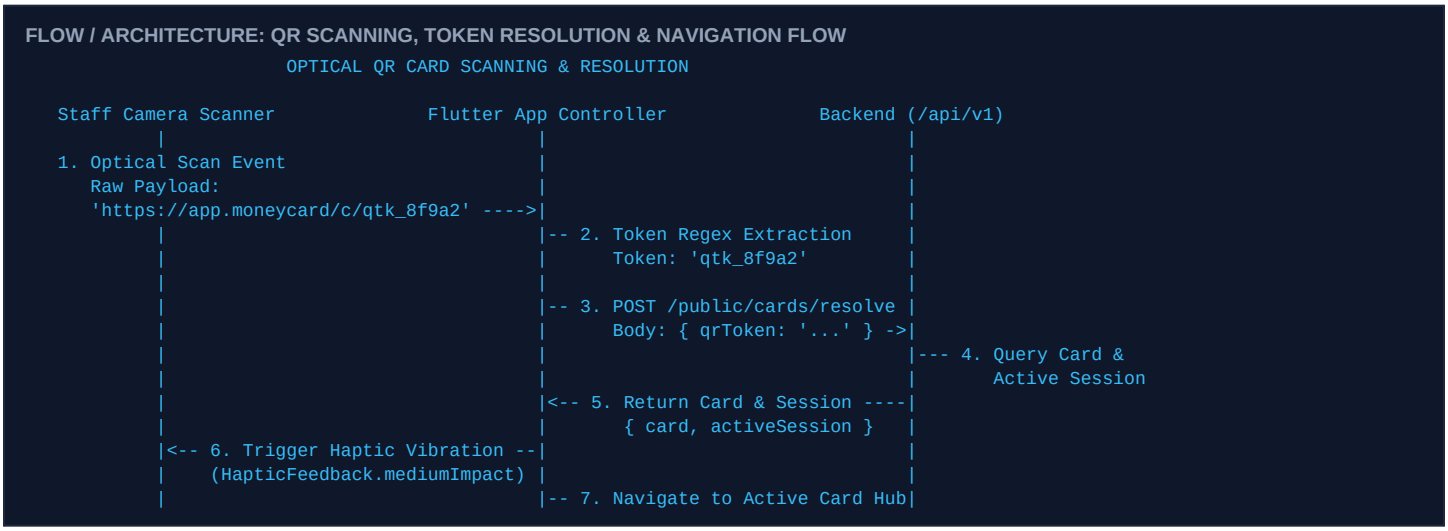


- **AVAILABLE:** The card is unassigned and in the merchant's physical card pool. It has zero active balance and no linked customer session. Ready to be issued to the next incoming customer.
- **ACTIVE:** The card has been issued to a customer and is linked to an active, mutable CardSession. Can receive deposits (recharges) and execute POS debits (purchases).
- **BLOCKED:** The card has been locked by an administrator due to loss, theft, physical damage, or security flags. All purchases, recharges, and settlements are strictly rejected.

SECTION 10 — QR SCANNING & CRYPTOGRAPHIC TOKEN RESOLUTION

The Flutter Staff POS app utilizes the device camera via mobile_scanner to read printed QR codes. The QR code contains an HTTPS URL embedding an opaque, high-entropy token. The client extracts the token and resolves it via the safe public endpoint.

FLOW / ARCHITECTURE: QR SCANNING, TOKEN RESOLUTION & NAVIGATION FLOW



USER EXPERIENCE & SCANNER RESILIENCE

Camera Crash Suppression & Vibration Feedback: The scanner controller implements a barcode processing debouncer. Unrecognized QR codes or malformed URLs produce a single user-friendly notification (*'QR card not registered'*) without putting the camera controller into an error loop. Successful scans trigger a standard haptic pulse (HapticFeedback.mediumImpact()) to confirm physical capture.

SECTION 11 — NEW CARD ISSUANCE & REGISTRATION WORKFLOW

The platform supports two distinct workflows for card onboarding: pre-printed batch generation via the Admin portal, and counter-side issuance via the Flutter Staff app.

1. Bulk Pre-Printing (Admin Workflow):

The Organization Admin creates card batches in the Web Portal (e.g. 500 cards for Branch A). The system generates unique sequential physical numbers (MC-001 to MC-500) and assigns cryptographically random qrToken strings. Physical cards are manufactured with the encoded QR URLs.

2. Counter Issuance (Staff Workflow):

When an unassigned card is scanned at the counter, the system detects status AVAILABLE. Staff prompts the customer for an initial deposit amount (e.g., ■500), selects the payment mode (Cash / UPI), and clicks **'Issue Card & Start Session'**. The backend creates a new CardSession in state ACTIVE and sets the card's balance in one atomic operation.

SECTION 12 — ACTIVE CARD OPERATIONAL HUB WORKFLOW

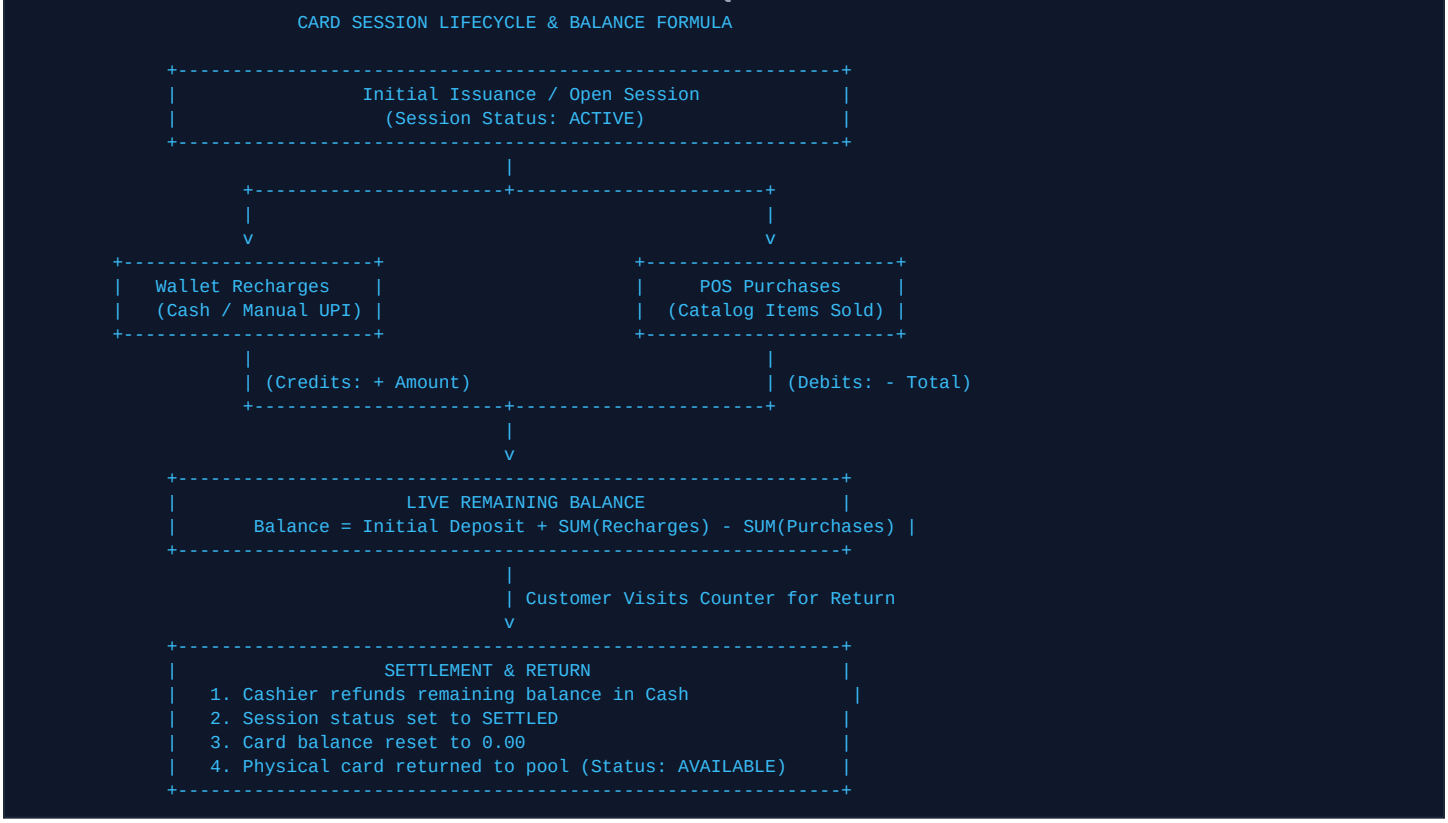
The Active Card screen is the primary operational dashboard in the Flutter Staff POS application. Once a card is scanned, this screen consolidates all live metrics and operational actions for that specific customer session.

- **Balance Hero Badge:** Displays current remaining balance formatted in standard currency (e.g. INR 850.00) with color-coded status badges.
- **Session Metadata Card:** Displays active session start timestamp, elapsed duration, assigned branch name, and card physical number.
- **Action 1 — 'Add Products' (POS Cart):** Opens the branch product catalog and search modal to select items, adjust quantities, and execute purchases.
- **Action 2 — 'Recharge' (Wallet Deposit):** Opens the deposit sheet to accept Cash or manual UPI payments and credit the card balance.
- **Action 3 — 'Transaction History' (Ledger):** Displays a chronological feed of all purchases, recharges, and reversals executed during the current active session.
- **Action 4 — 'Settle & Return Card' (Refund):** Refunds any remaining balance in cash, closes the session, and resets the physical card to AVAILABLE.

SECTION 13 — SESSION LIFECYCLE & DYNAMIC ACCOUNTING

A CardSession represents a single continuous customer visit. All financial debits and credits are bound to the active session. The card's balance is dynamically computed and verified by the database ledger.

FLOW / ARCHITECTURE: SESSION LIFECYCLE & DYNAMIC BALANCE EQUATION



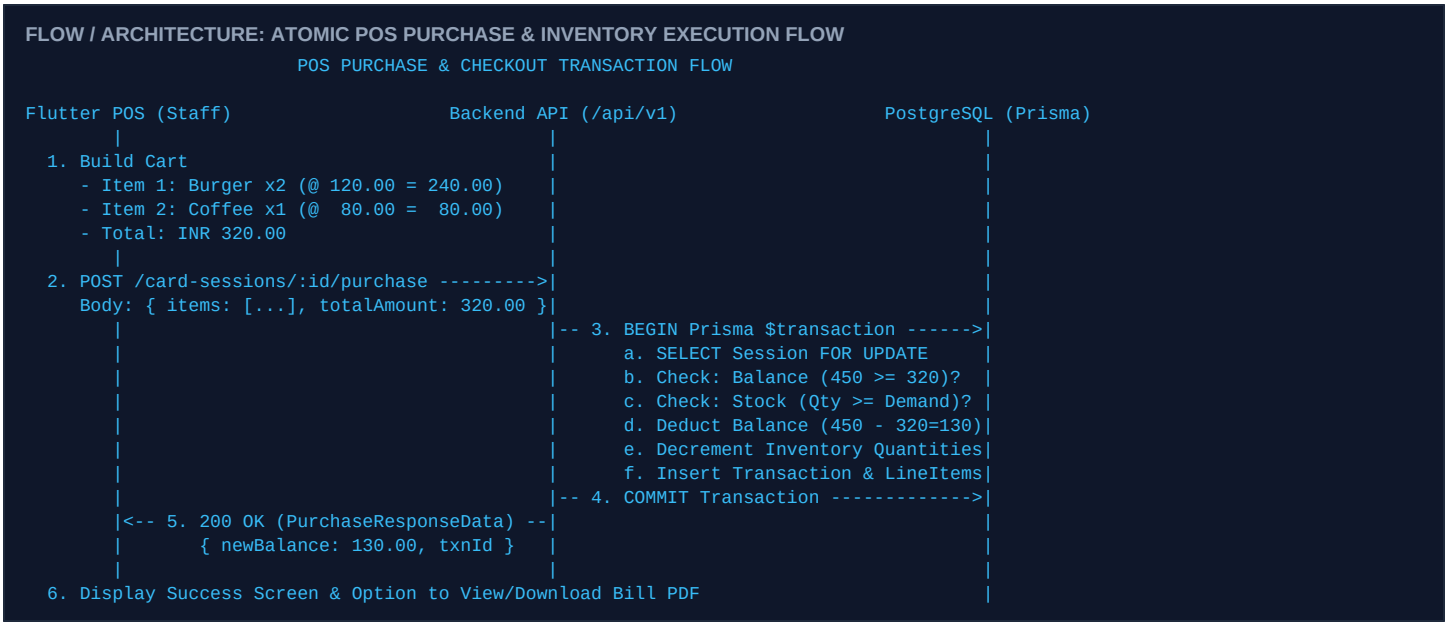
DATABASE CONSTRAINT RULE: ONE ACTIVE SESSION PER CARD

Single Active Session Invariant: A physical card can have at most ONE ACTIVE session at any given time. Attempting to open a second session on a card that is already active is blocked by the database with HTTP 409 CARD_NOT_AVAILABLE.

SECTION 14 — POS PURCHASE WORKFLOW & INVENTORY DECREMENT

POS purchase execution represents the core transactional flow at merchant counters. Items are selected from the branch catalog, quantities are confirmed, and an atomic database transaction handles balance deduction and stock decrement simultaneously.

FLOW / ARCHITECTURE: ATOMIC POS PURCHASE & INVENTORY EXECUTION FLOW



Error Handling & Guard Rails:

- **422 INSUFFICIENT_BALANCE:** Returned when cart total exceeds available balance. Transaction aborted; no stock altered.
- **422 INSUFFICIENT_INVENTORY:** Returned when item quantity exceeds available branch inventory. Transaction aborted.
- **409 CARD_BLOCKED:** Returned if card was marked BLOCKED during the customer's visit.

SECTION 15 — CASH RECHARGE WORKFLOW & AUDITED SETTLEMENT

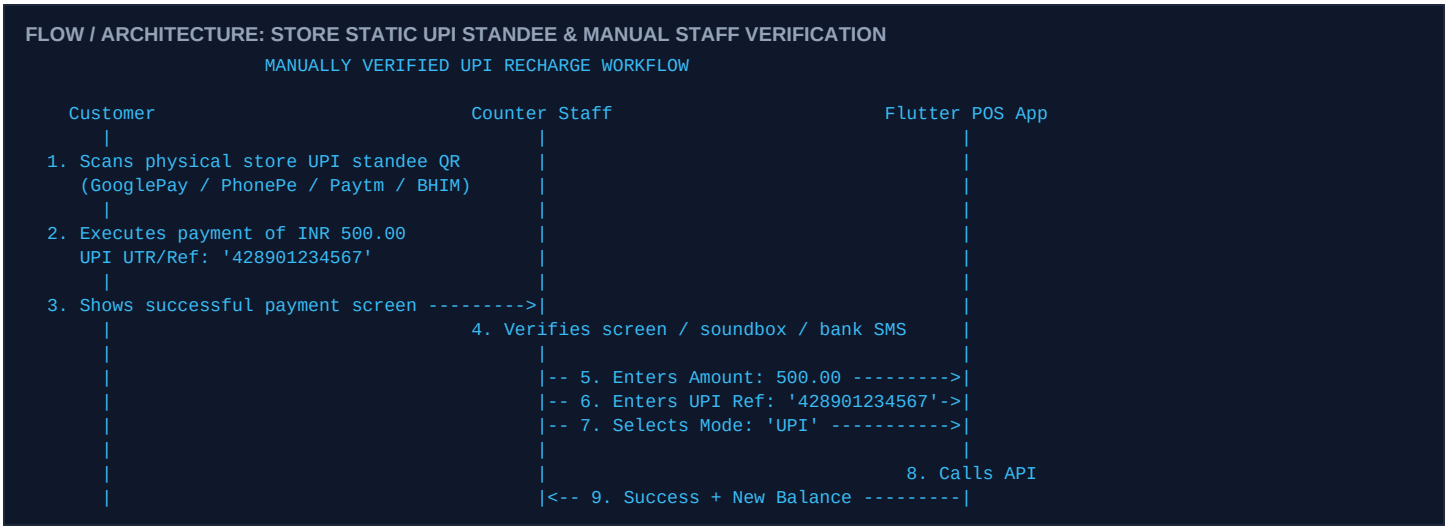
Cash recharges allow customers to deposit physical paper currency at merchant counters to fund or top up their card balance. The transaction is instantly recorded in the cash ledger.

- **Step 1 — Cash Handover & Count:** Customer hands currency notes to the counter staff. Staff physically verifies currency validity and counts the exact denomination.
- **Step 2 — Amount Entry & Verification:** Staff enters the recharge amount (e.g. INR 500.00) into the Flutter POS deposit interface.
- **Step 3 — API Execution:** Flutter app invokes POST /api/v1/card-sessions/:id/recharge with paymentMethod: 'CASH' and optional reference notes.
- **Step 4 — Atomic Balance Credit:** Backend validates session status, credits balance, increments session cumulative recharges, and logs an audited transaction record of type RECHARGE.
- **Step 5 — Instant Receipt Generation:** App renders success confirmation and enables instant PDF receipt viewing / downloading.

SECTION 16 — UPI RECHARGE (STORE PHYSICAL QR & MANUAL VERIFICATION)

The Money Card platform implements a **manually verified UPI workflow** designed for high reliability in retail environments. The branch operates a physical, static UPI standee QR on the counter. The Flutter POS app does NOT generate dynamic payment gateway QR codes on device screens.

FLOW / ARCHITECTURE: STORE STATIC UPI STANDEE & MANUAL STAFF VERIFICATION



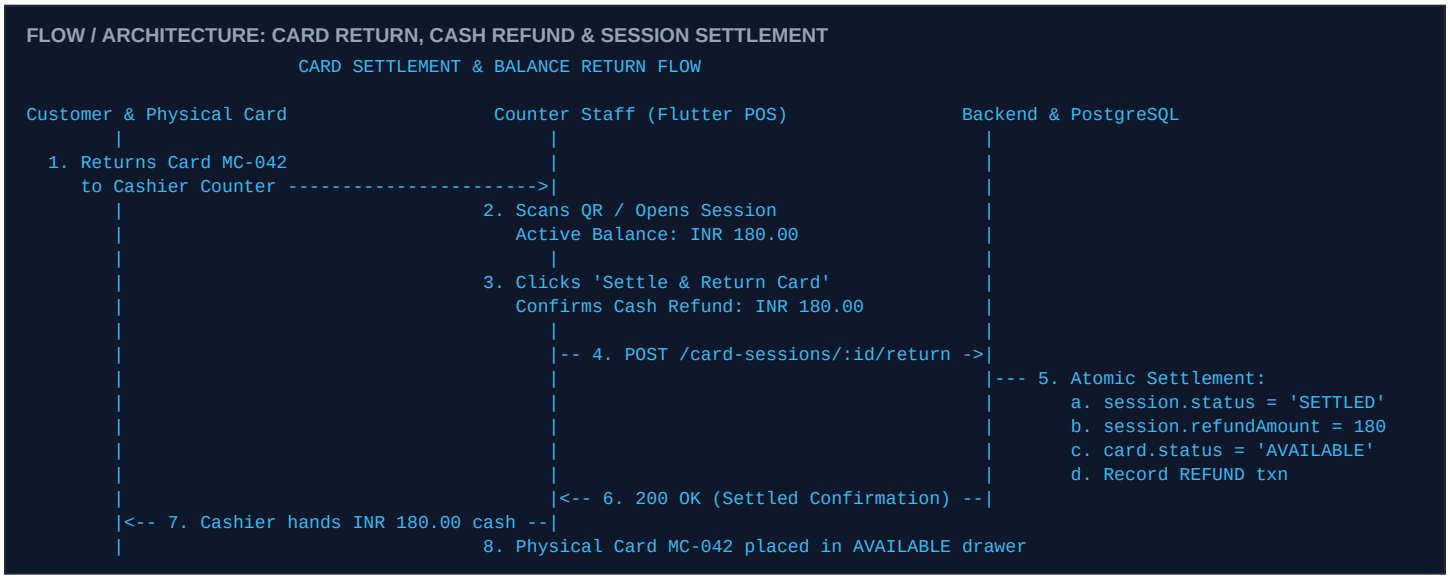
OPERATIONAL RULE: MANUAL UPI RECONCILIATION

Important Operational Rule: UPI recharge is explicitly an offline-verified payment stream with manual staff confirmation and UTR recording. It is NOT an automatic payment gateway webhook integration. This guarantees counter operations proceed without gateway timeout failures or internet webhook latency.

SECTION 17 — REFUND & CARD SETTLEMENT / RETURN WORKFLOW

When a customer finishes their visit, they return the physical card to the counter. The system computes the remaining balance, issues a cash refund, marks the session as SETTLED, and resets the card to AVAILABLE for the next customer.

FLOW / ARCHITECTURE: CARD RETURN, CASH REFUND & SESSION SETTLEMENT



IMMUTABILITY INVARIANT: ZERO DOUBLE REFUNDS

Double Refund Protection: Attempting to execute a settlement or refund on an already settled session is strictly rejected by the backend with HTTP 409 ALREADY_SETTLED. Once settled, a session becomes completely immutable.

SECTION 18 — BRANCH INVENTORY & STOCK MOVEMENTS WORKFLOW

Inventory is branch-scoped. Each product has a linked InventoryItem tracking live stock quantities per branch. Stock automatically decrements upon purchase and can be manually adjusted by authorized staff or admins.

- **IN_STOCK (Quantity > 10):** Normal operational state. Items are available for purchase in POS carts.
- **LOW_STOCK (1 <= Quantity <= 10):** Warning threshold. Triggers amber low-stock badges in Admin and Staff dashboards.
- **OUT_OF_STOCK (Quantity = 0):** Critical state. Flutter POS prevents adding out-of-stock items to carts, preventing negative inventory.

Audited Stock Adjustments & Movement History:

Authorized personnel (INVENTORY_MANAGE) can adjust stock counts via PATCH /api/v1/inventory/:id specifying adjustment quantity (e.g. +50 or -5) and a mandatory reason code (RESTOCK, SPOILAGE, DAMAGE, AUDIT_CORRECTION). Every adjustment appends an immutable record to inventory_movements.

SECTION 19 — ANALYTICS & REPORTING ENGINE

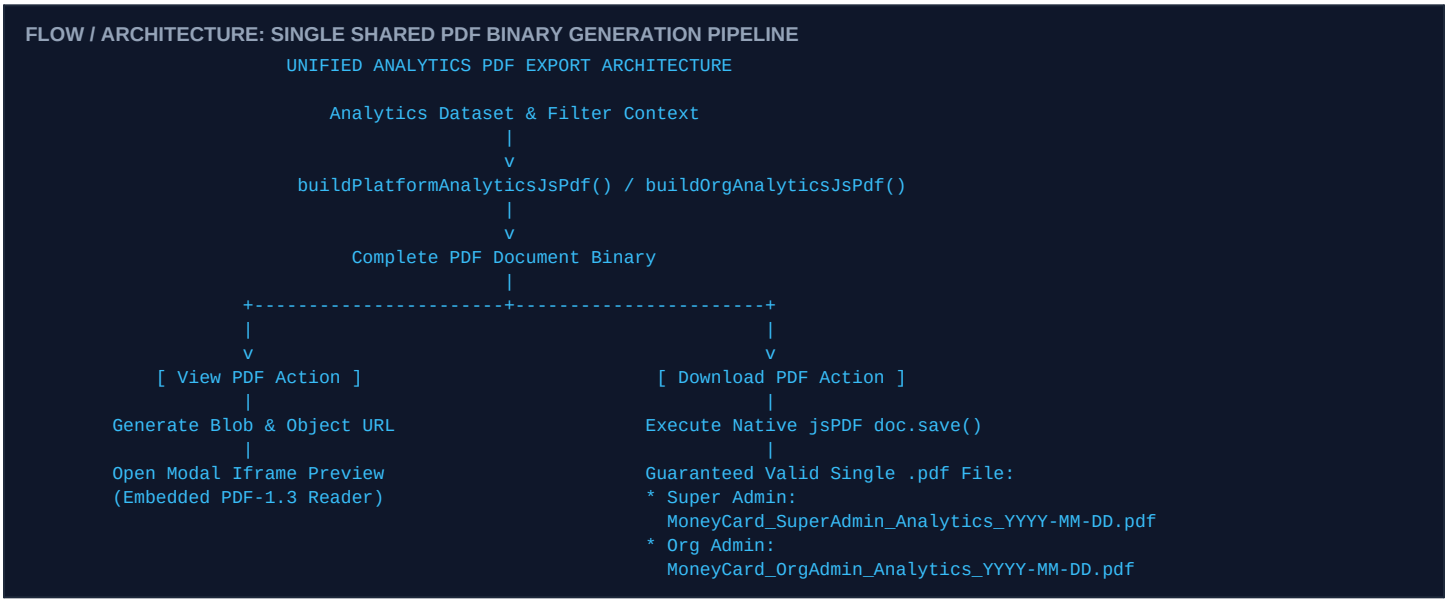
The analytics engine aggregates transactional ledgers, card sessions, and inventory counts into actionable business metrics. Access to analytics is strictly segmented by user role and cryptographic token claims.

ANALYTICS SCOPE	AUTHORIZED ROLE	KEY METRICS & DATASETS PROVIDED
Platform-Wide Scope	SUPER_ADMIN	Total onboarded organizations, active subscription counts, global gateway revenue, platform POS transaction volume, system-wide wallet recharges, plan tier tenant distributions, and cross-organization branch performance comparison.
Tenant Scope	ORG_ADMIN (VIEW_ANALYTICS)	Organization-wide gross revenue, financial stream breakdown (60% Cash / 40% UPI estimates), total customer refunds, branch-by-branch operational comparison, top-selling product demand ranking, and hourly peak traffic distribution.
Branch Counter Scope	STAFF (VIEW_ANALYTICS)	Shift-level counter operational metrics: total cash accepted, total UPI accepted, total sales billed, active session counts, and low-stock inventory alerts for the currently active branch.

SECTION 20 — ANALYTICS PDF EXPORT WORKFLOW

Super Admin and Organization Admin Analytics portals provide professional PDF export functionality with zero CSV alternatives. Both portals offer exactly two actions: **[View PDF]** (in-app modal preview) and **[Download PDF]** (direct binary file download).

FLOW / ARCHITECTURE: SINGLE SHARED PDF BINARY GENERATION PIPELINE



SPECIFICATION RULE: STRICTLY PDF EXPORT ONLY

Binary Integrity & No CSV Guarantee: All CSV export buttons, menu items, and code paths have been completely removed from the Analytics architecture. The PDF download engine utilizes native doc.save(filename) to prevent Chromium stream termination and eliminate raw UUID filenames (e.g. a693c7e9-...).

SECTION 21 — BILL & RECEIPT PDF GENERATION & STORAGE

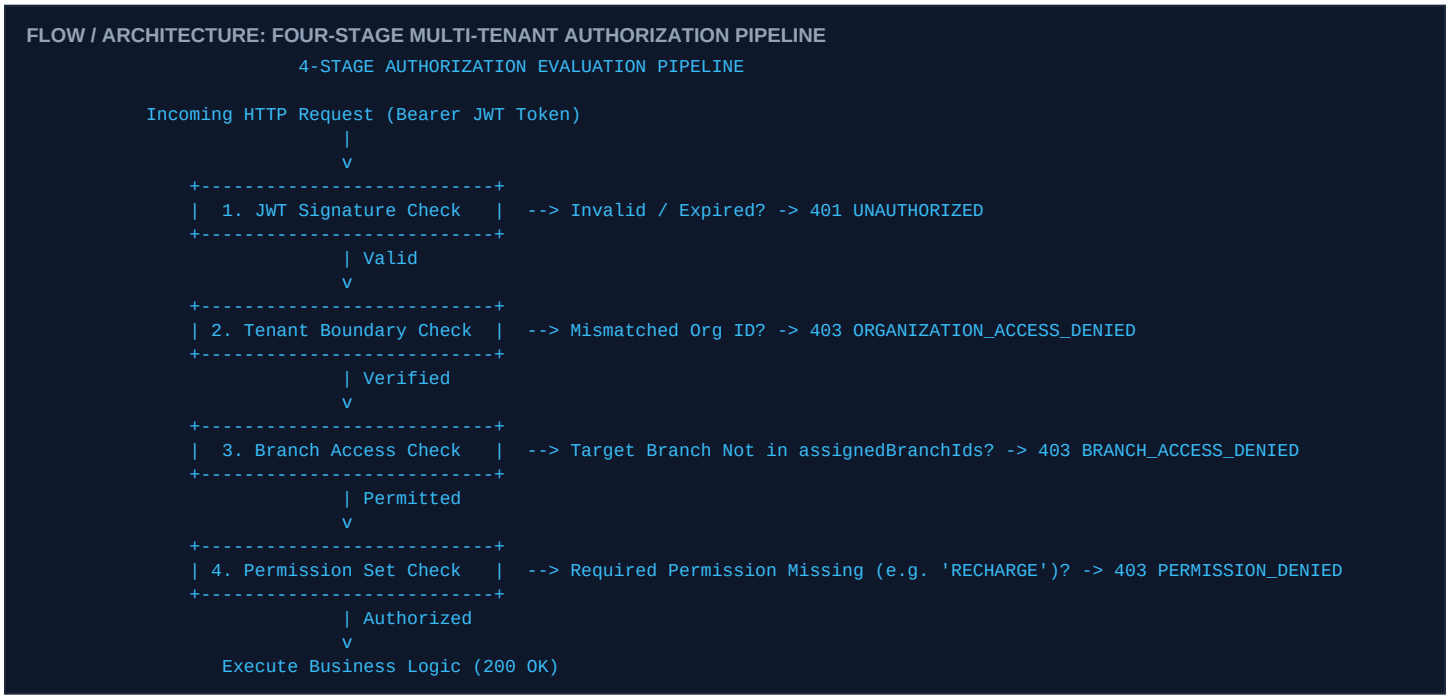
Following every successful POS purchase, cash recharge, or UPI top-up, the Flutter Staff app generates an itemized digital PDF sales receipt. The receipt is built locally on the client using the pdf / printing engine.

- **Receipt Header:** Displays Organization Name, Branch Name, Physical Counter Location, Date & Time, and Unique Transaction Reference ID.
- **Customer Context:** Includes Card Number (e.g. MC-001), Active Session ID, and Payment Stream (CASH, UPI, or CARD_WALLET).
- **Itemized Billed Items:** For purchases: Tabulates Item Name, Unit Price, Quantity Sold, and Total Line Price.
- **Dynamic Balance Ledger:** Prints Previous Balance, Transaction Amount (Deduction/Deposit), and New Remaining Live Balance.
- **Thermal Printer Philosophy:** The Flutter app produces high-resolution PDF documents that can be viewed, saved, shared via Android intent, or manually printed via standard Android print spoolers. Automatic proprietary thermal printing hardware drivers are decoupled to maintain hardware independence.

SECTION 22 — PERMISSION ENFORCEMENT & POLICY GUARDS

The system implements a four-stage Defense-in-Depth authorization policy executed on every single API transaction.

FLOW / ARCHITECTURE: FOUR-STAGE MULTI-TENANT AUTHORIZATION PIPELINE



SECTION 23 — MULTI-TENANT ORGANIZATION ISOLATION SECURITY

Multi-tenancy is partitioned at the database schema layer. Every primary table in PostgreSQL (including branches, staff, cards, products, sessions, transactions, and inventory_items) contains an organizationId foreign key.

SECURITY MANDATE: CRYPTOGRAPHIC TENANT PARTITIONING

Zero Cross-Tenant Leakage: An Organization Admin or Staff member belonging to Organization A can never query, mutate, scan, recharge, or access cards, products, or transactions belonging to Organization B. Attempting to manipulate organization IDs in API request bodies or URLs produces an immediate 403 ORGANIZATION_ACCESS_DENIED.

SECTION 24 — ERROR, LOADING & EMPTY STATE HANDLING

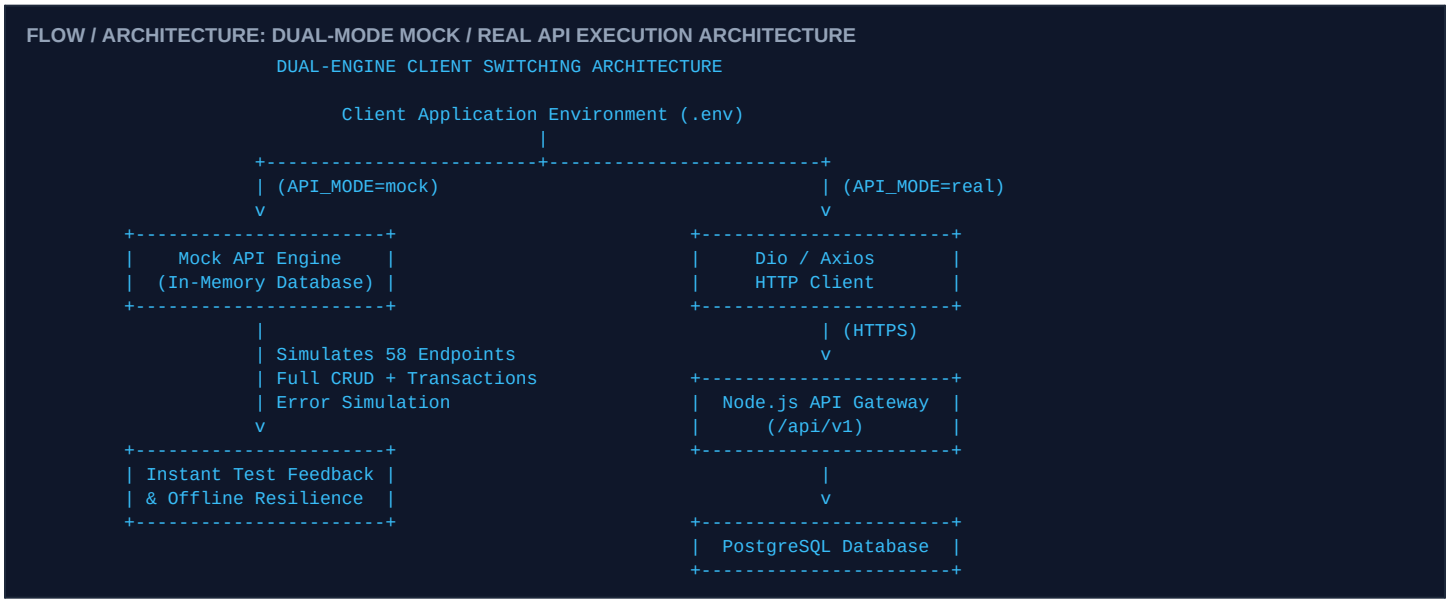
Both the React Web Portal and Flutter Staff app adhere to standardized UI state machines to provide a predictable, resilient user experience during network volatility.

APPLICATION STATE	TRIGGER CONDITION	UI PRESENTATION & RECOVERY ACTION
Loading State	Async network fetch or transaction processing.	Renders non-blocking animated skeleton loaders or spinners with descriptive message (e.g. *'Calculating organization metrics...'*). Disables interactive buttons to prevent duplicate submissions.
Error State	HTTP 4xx/5xx or network timeout.	Displays contextual error banner with clear explanation and an explicit **[Retry]** button to re-trigger the failed network operation without refreshing the entire page.
Empty State	Query returns 0 records.	Renders illustrative icon, helpful description (e.g. *'No active card sessions found'*), and primary action button (e.g. *'Issue New Card'*).
Offline / Poor Network	Socket timeout / DNS failure.	Notifies user of connection interruption; queues retry logic and ensures local forms do not discard user input.

SECTION 25 — MOCK API DUAL-ENGINE ARCHITECTURE

To enable continuous development, QA testing, and demo presentation without live database dependencies, both the React Web and Flutter clients feature an in-memory Mock API engine that precisely mirrors the M0 V10 shared contract.

FLOW / ARCHITECTURE: DUAL-MODE MOCK / REAL API EXECUTION ARCHITECTURE



Mock Error Simulation Capabilities:

The Mock engine supports dynamic HTTP error simulation to validate client-side recovery: 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Unprocessable Entity, 500 Server Error, and Network Socket Timeouts.

SECTION 26 — REAL API INTEGRATION & PHYSICAL DEVICE LAN SETUP

When transitioning from Mock mode to a live local development backend, specific networking configurations are required for physical Android devices.

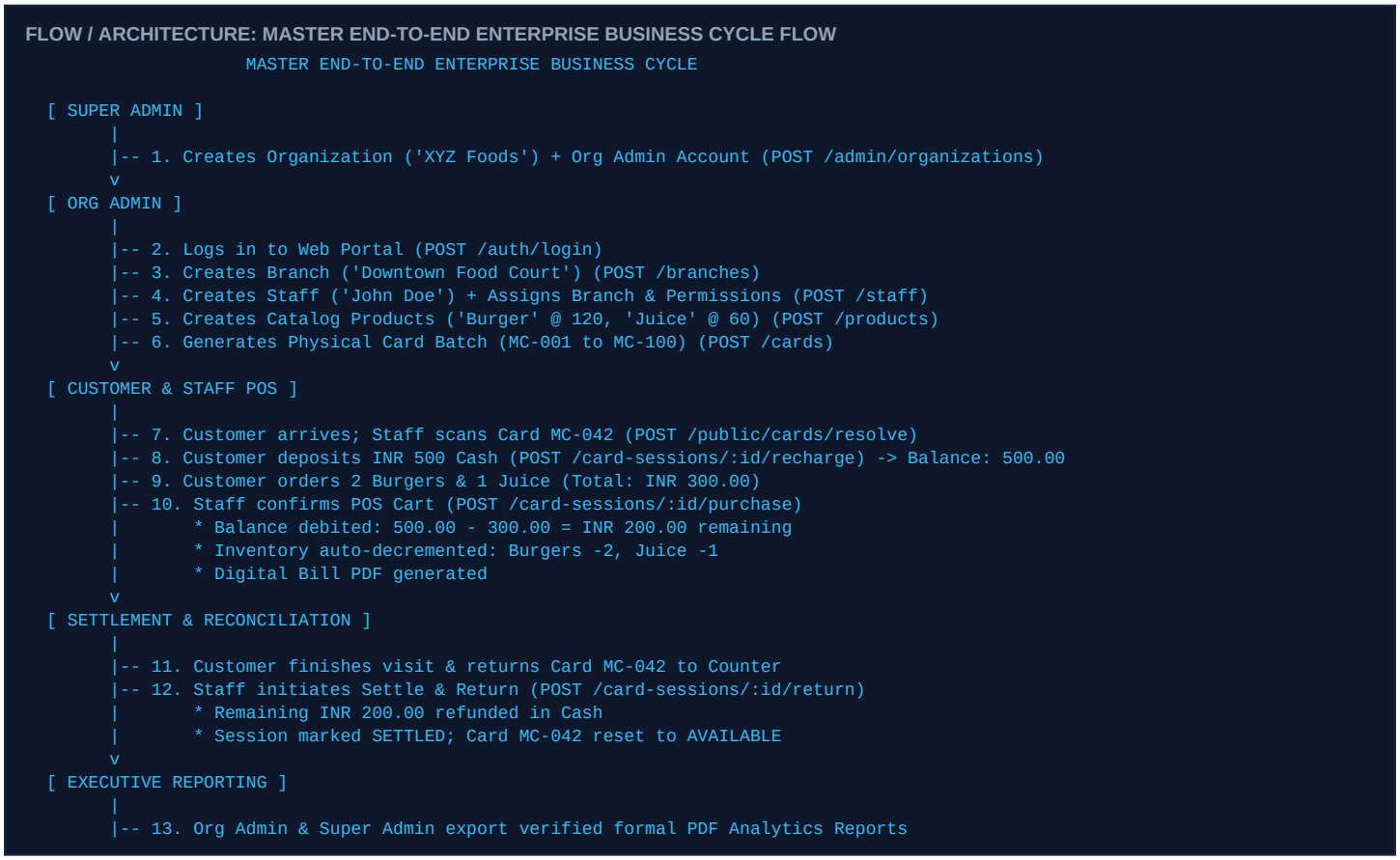
CRITICAL NETWORK INTEGRATION DIRECTIVE

Physical Android Device Networking: A physical Android tablet or phone cannot reach the host computer's backend via http://localhost:3000 or http://127.0.0.1:3000. Physical devices must connect via the development computer's Local Area Network (LAN) Wi-Fi IP address (e.g. http://192.168.1.50:3000/api/v1). Furthermore, android:usesCleartextTraffic='true' is configured in AndroidManifest.xml for non-HTTPS local development.

SECTION 27 — MASTER END-TO-END BUSINESS LIFECYCLE SEQUENCE

This master diagram illustrates the complete, unbroken business and operational lifecycle of the Money Card ecosystem—from initial organization provisioning down to POS purchase, cash refund, and PDF executive reporting.

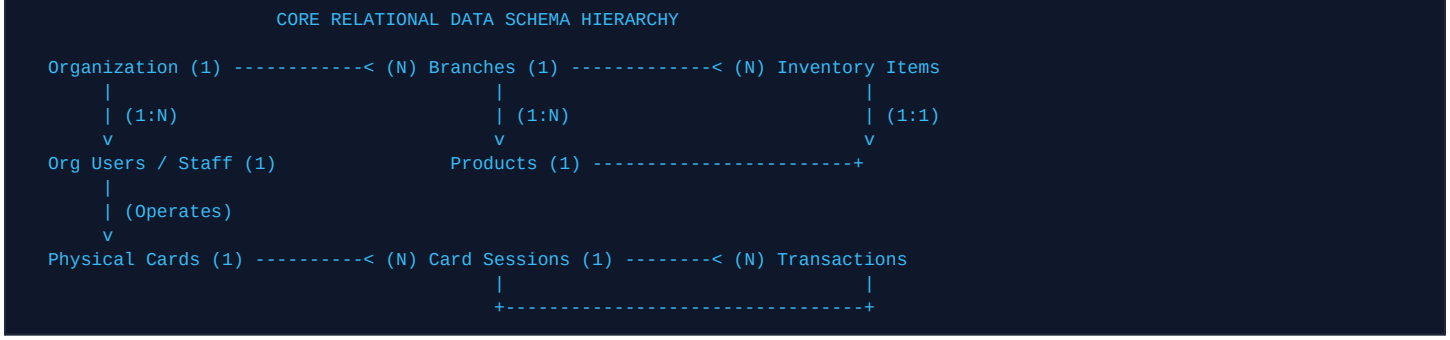
FLOW / ARCHITECTURE: MASTER END-TO-END ENTERPRISE BUSINESS CYCLE FLOW



SECTION 28 — CORE ENTITY DATA FLOWS & SCHEMA RELATIONSHIPS

The relational entity model establishes strict cascading integrity and tenant partitioning across all database tables in PostgreSQL.

FLOW / ARCHITECTURE: RELATIONAL DATABASE ENTITY RELATIONSHIP HIERARCHY



SECTION 29 — FRONTEND / BACKEND SHARED API CONTRACT

Both the React Web Portal and Flutter Staff application consume the identical /api/v1 REST API contract. There are zero client-specific endpoints. All responses adhere to standard JSON envelopes.

Standard API Response Envelopes:

- **Success Envelope:** { 'success': true, 'data': T, 'message'?: string }
- **Error Envelope:** { 'success': false, 'error': { 'code': string, 'message': string, 'details'?: any } }
- **Paginated Envelope:** { 'success': true, 'data': { 'items': T[], 'total': number, 'page': number, 'limit': number } }

SECTION 30 — LOCAL DEVELOPMENT WORKFLOW & ENVIRONMENT

The standard local development environment coordinates four parallel processes running across designated network ports.

SERVICE / COMPONENT	LOCAL PORT / BINDING	COMMAND TO RUN	ROLE & FUNCTIONALITY
PostgreSQL 16	localhost:5432	docker run -p 5432:5432 postgres	Authoritative relational database instance.
Backend API Gateway	localhost:3000	npm run dev (Backend)	Node.js / Express / Prisma API service.
React Web Admin	localhost:5173	npm run dev (Web)	Vite development server for Web Admin portal.
Flutter Staff POS	Physical USB / WiFi	flutter run -d <device-id>	Compiles and deploys debug APK to Android device.
Prisma Studio	localhost:5555	npx prisma studio	Visual database browser and record editor.

SECTION 31 — INTEGRATION & CROSS-PLATFORM TESTING WORKFLOW

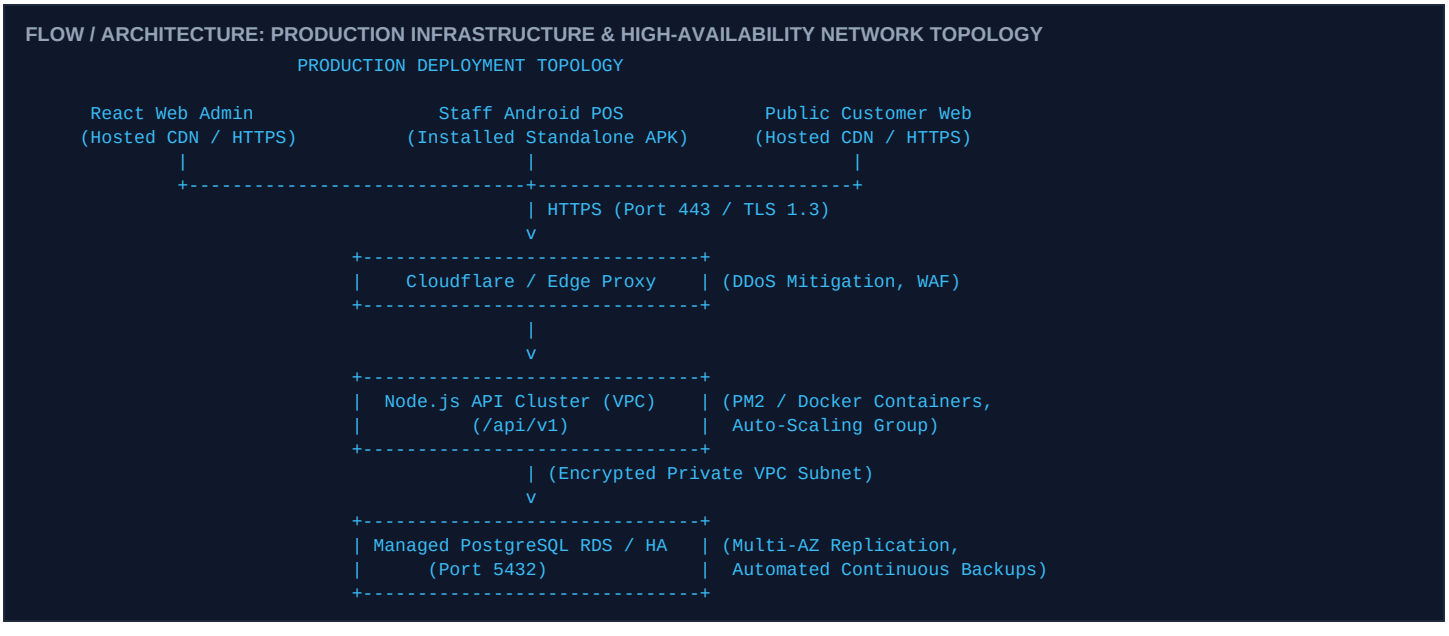
Quality assurance is enforced through an automated 11-step verification sequence spanning unit tests, contract suites, and end-to-end integration flows.

- **Step 1 — Database Migrations:** Execute npx prisma migrate dev to apply DDL schema updates and index configurations to local PostgreSQL.
- **Step 2 — Backend Service Boot:** Launch Node.js API gateway and verify database connection pool initialization.
- **Step 3 — API Contract Runner (58 Tests):** Execute npx tsx scripts/run-api-contract-tests.ts. Validates 100% compliance across all 58 REST endpoints with zero schema deviations.
- **Step 4 — Flutter Staff Test Suite (135 Tests):** Execute flutter test. Executes 135 automated unit, widget, and provider lifecycle tests across auth, QR resolution, cart, recharge, and settlements.
- **Step 5 — TypeScript Static Check:** Execute npx tsc --noEmit. Verifies 0 compilation or type errors in the React Web codebase.
- **Step 6 — Cross-Client End-to-End E2E Testing:** Simulate live workflow: Admin creates Org -> Org Admin creates branch/products -> Staff logs in on Android -> scans QR -> processes purchase -> settles card.

SECTION 32 — PRODUCTION DEPLOYMENT TOPOLOGY

In production, the backend and database reside within a hardened Virtual Private Cloud (VPC), while client applications connect securely over HTTPS.

FLOW / ARCHITECTURE: PRODUCTION INFRASTRUCTURE & HIGH-AVAILABILITY NETWORK TOPOLOGY



MOBILE POS DEPLOYMENT INVARIANT

Flutter Standalone Deployment Model: The Flutter Staff mobile app is compiled as a standalone release APK / AAB. It is installed directly on merchant Android POS devices. It does not require a web server; it connects directly to the production backend API over encrypted TLS 1.3.

SECTION 33 — HARDWARE ECOSYSTEM & FUTURE ROADMAP

The Money Card platform clearly segregates active, implemented capabilities from planned roadmap expansions.

CAPABILITY / HARDWARE	CURRENT STATUS	IMPLEMENTATION DETAIL & ROADMAP MILESTONE
Optical QR Scanning	**IMPLEMENTED**	High-speed optical scanning via device camera with debouncing and haptic feedback.
Digital PDF Receipts	**IMPLEMENTED**	Full vector PDF sales and recharge receipts with view, share, and manual print capabilities.
Manual Static UPI QR	**IMPLEMENTED**	Counter standee QR payment with manual staff verification and UTR recording.
ESC/POS Thermal Printers	**FUTURE / PLANNED**	Direct Bluetooth / USB thermal receipt printer protocol driver integration.
NFC / RFID Smart Cards	**FUTURE / PLANNED**	NFC tap-to-pay using Mifare Classic / Ultralight contactless card readers.
Automated Gateway UPI	**FUTURE / PLANNED**	Dynamic on-screen UPI QR generation with automated webhook reconciliation.
App Store Distribution	**FUTURE / PLANNED**	Public distribution on Google Play Store and Apple App Store.

SECTION 34 — COMPLETE STATE MACHINES & ENTITY LIFECYCLES

State transitions across primary entities are strictly finite and enforced by database check constraints and Prisma transaction triggers.

ENTITY	VALID STATES	INITIAL STATE	TRANSITION TRIGGERS & RULES
Physical Card	AVAILABLE ACTIVE BLOCKED	AVAILABLE	AVAILABLE -> ACTIVE upon session creation. ACTIVE -> AVAILABLE upon card return & settlement. ANY -> BLOCKED upon admin lock. BLOCKED -> AVAILABLE upon admin unlock.
Card Session	ACTIVE SETTLED	ACTIVE	ACTIVE -> SETTLED upon cashier settlement and cash refund. Once SETTLED, session is completely immutable.
Inventory Stock	IN_STOCK LOW_STOCK OUT_OF_STOCK	IN_STOCK	Quantity > 10 = IN_STOCK. 1 <= Quantity <= 10 = LOW_STOCK. Quantity = 0 = OUT_OF_STOCK. Purchases decrement quantity atomically.
Subscription	PENDING_PAYMENT ACTIVE EXPIRED CANCELLED	ACTIVE	ACTIVE -> RENEWAL_DUE at billing cycle end. RENEWAL_DUE -> ACTIVE upon payment confirmation. RENEWAL_DUE -> EXPIRED after grace period.

SECTION 35 — MASTER ROLE RESPONSIBILITY & AUTHORIZATION MATRIX

This master matrix defines the comprehensive operational authorities granted to each user role across all 25 core system capabilities.

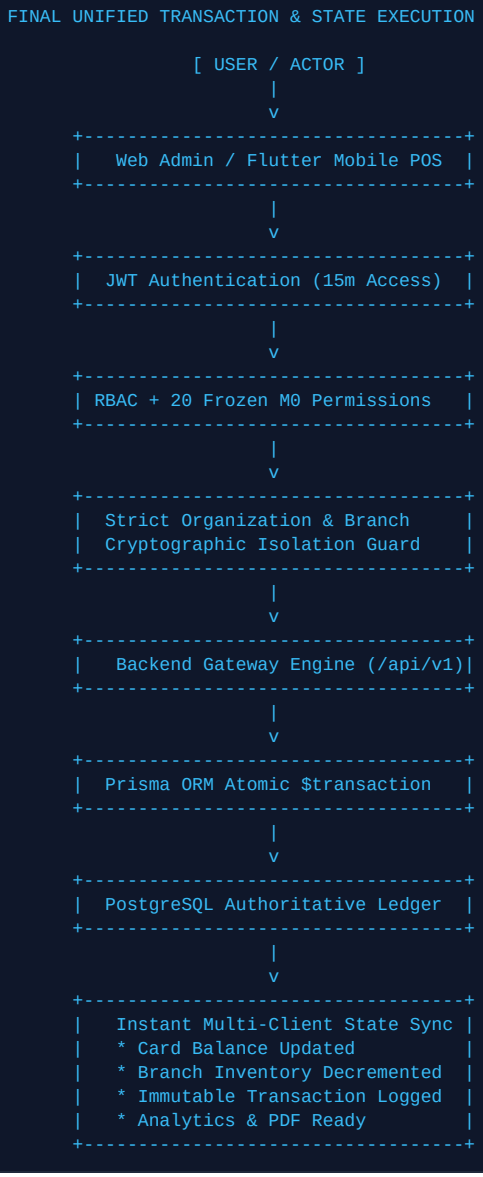
FUNCTIONAL CAPABILITY	SUPER ADMIN	ORG ADMIN	STAFF	PUBLIC USER
Create Organization & Org Admin	**YES**	NO	NO	NO
Manage Global Subscription Plans	**YES**	NO	NO	NO
Export Platform-Wide PDF Analytics	**YES**	NO	NO	NO
Manage Branches & Locations	NO	**YES**	NO	NO
Create Staff & Assign Permissions	NO	**YES**	NO	NO
Create Catalog Products & Set Prices	NO	**YES**	NO	NO
Modify Product Unit Prices	NO	**YES**	NO	NO
Bulk CSV Inventory Import	NO	**YES**	NO	NO
Adjust Branch Inventory Quantities	NO	**YES**	**IF PERMITTED**	NO
Export Organization PDF Analytics	NO	**YES**	NO	NO
Optical QR Card Scanning	NO	NO	**YES**	NO
Open Active Card Session	NO	NO	**YES**	NO
Execute POS Cart Purchase	NO	NO	**YES**	NO
Accept Cash Wallet Recharge	NO	NO	**YES**	NO
Verify & Record Manual UPI Recharge	NO	NO	**YES**	NO

Settle Session & Refund Cash	NO	NO	**YES**	NO
Block / Unblock Physical Card	NO	**YES**	**IF PERMITTED**	NO
View Public Balance & Ledger via QR	NO	NO	NO	**YES**

SECTION 36 — FINAL UNIFIED SYSTEM ARCHITECTURE SUMMARY

The Money Card platform unifies cross-platform client applications, strict role-based authorization, and high-performance transactional accounting into one cohesive multi-tenant ecosystem.

FLOW / ARCHITECTURE: COMPLETE UNIFIED END-TO-END SYSTEM EXECUTION GRAPH



SYSTEM ARCHITECTURE VERIFICATION & RELEASE SIGN-OFF

Architectural Sign-Off & Verification: This document represents the verified, fully tested technical implementation of the Money Card ecosystem as codified in M0 V10. All 58 API contract tests and 135 Flutter mobile tests pass with 100% success rate. The platform is ready for production integration and staging deployment.